CS341 CacheLab Part B Report

Tomas Salaz, Rafael Campos Chavarria

December 2025

Abstract

For this project, we wanted to explore the differences in performance between AI code and Human Code when it comes to transposing Matrices and optimizing those functions. We claim that AI is a powerful programming tool that greatly improves productivity in the hands of a programmer who has strong understandings and skills.

Introduction and Motivation

Cache management matter for Matrix Transposing. In this project, we explored the difference in performance between AI generated code from `ChatGPT-5.1` and Human written code. We will focus firstly on the methodology used and the data we collected from both sides. Then we will compare the results and analyze the two implementations.

Methods

The project asked us to transpose 2 square matrices of sizes `32x32` and `64x64`, as well as a `67x61` matrix. We wanted to implement the code to be optimized to have as few cache misses as possible.

AI Generated Code

For the AI portion of the project, we used `ChatGPT-5.1`. We attached the skeleton code for `trans.c` and kept the prompts very simple and outlined 3 guidelines for the AI to follow; Problem, Restrictions, Desired performance. The prompt is as follows

```
Use the attached .c file and implement code to follow
these guidlines.
Task: Immplement matric matricx multiplications, we are testing
on 3 matrix sizes: 32x32, 64x64 and 61x67.
```

```
Restrictions: *Copied and Pasted Restrictoins from PDf*
Performance: *Copied and Pasted Rubric section with Miss limits*
```

`ChatGPT-5.1` took 3 minutes to generate code and, which we were able to copy and paste into `trans.c`.

Human Code

A really important thing when it comes to cache friendly code is temporal locality. To increase temporal locality we will be using the concept of Blocking, which was introduced to us via the provided article. To choose the optimal block size, we need to look at the cache parameters. We are told that the cache is direct mapped, s = 5, and b = 5. This tells us that the cache has 32 sets and the block size is 32 bytes. This means that we can fit 8 integers in each cache line. For this reason, we will be using a block size of `8x8`.

Another important thing to know is that matrices are stored in row major order in memory. We can use this information to get an idea of which addresses are assigned to which sets. This is very important because it lets us see which address will cause conflicts and evictions in the sets.

Looking at the case of transposing a `32x32` matrix, we can calculate that 4 sets fit in the first row. This tells us that the first 8 rows map to all 32 different sets. This means that each row in the 8 * 8 block will map to a different set. We will keep this in mind when deciding the order in which we access matrix A and B.

When looking at the case of transposing a `64*64` things get more complicated. We calculate that 8 sets will fit in the first row which means that the first 4 rows map to all 32 different sets. This means that the top half and the bottom half of each `8x8` block will cause cache conflicts with each other. This makes reducing the cache misses a lot more complicated.

For the implementation, we made sure to use the `8x8` Blocks for all sizes so we can get the best cache performance. Using the `8x8` blocks means that we have already used 8/12 of our allotted local variables, which is fine since we only need 3 loop iterators, for any size's implementation.

Data Collection

The data collected from the experiment is going to focus on 3 aspects: Time to get a result, Correctness and Performance, and Code Quality. The *.c* files with the source code are on canvas in the `.tar` file we turned in and can also be found in the appendix section at the end of this document.

AI Generated Code

The code from `ChatGPT-5.1` performed well. On it's first attempt, the model was able to generate a working solution in 2 minutes and 51 seconds. The code fit within the design constraints and passed the tests within the thresholds outlined in the assignment document.

The quality of the code is passable, the variable names are not defined well nor explained anywhere in the generated code. So the humans are tasked with defining the purpose of each variable, which is not hard in this simple experiment.

Another short coming of the generated code is that the comments don't do a good job at explaining what is happening. When we code, we try to make sure the code is easily understandable to other humans, so the comments usually give a sentence or 2 to explain the important parts of the code.

Other than those 2 deficiencies, the code works and is understandable after you read through it a time or 2. This makes it hard to nitpick since the code is not complex and every line is serving a purpose.

Human Code

The code we wrote to transpose matrices took us some time. Rapahel Coded the `32x32` and the `64x64` solutions. Tomas coded the `61x67` solution and oversaw the AI portion.

All together, we took about 3 hours between the both of us to code the solutions. We did have to do some debugging which extended the time. But overall, We felt like we could've been more efficient if we had coded in the same room since we could;ve helped each other.

Of course we fell like our code quality is better than the AI generated code. We allowed for more modularity by making methods for the different sizes, making the code easier to read and understand.

The code correctness is also good. The AI and the human generated code was able to be correct and fit within the requirments.

Exploratory Analysis

The case of transposing a `32x32` matrix was simple and did not take much time. However, it did require a good understanding of how the cache works and how to write code that is cache friendly. Even though it was simple, it did require us to simulate transposing a matrix to figure out the access patterns that would

reduce the cache misses.

The case of transposing a `64x64` was a lot more complicated and took a considerable amount of time. It took thinking outside the box and multiple different approaches to come up with an acceptable solution. The more efficient the approach was the more complicated the code would get. We ended up with a function that is overly complicated and hard to follow and didn't offer a big reduction in the misses compared to some other solutions we came up with.

Conclusion

The code generated by ChatGPT 5 was significantly better in terms of performance for the case of transposing a `64x64` matrix. It only produced 1,172 misses compared to ours that produced 1,460. The approach it came up with was also less complex and the code was easier to understand. The time it took for us to prompt it and for it to generate the code was significantly less than what it took for us to write it. For the other matrix our code performance was exactly the same and was of similar complexity.
This show us that AI is capable of producing equally efficient if not better code when solving a known problem like transposing a matrix in a cache friendly way. We believe that the reason it can do this is because this is a problem that has already been solved before and the solution to it can be found in the internet. It would be interesting to see how AI does at a more complicated and unique problem whose solution isn't available in the internet. None the less we conclude that AI is powerful tool but its even more powerful if the programmers who use it have strong programming understanding and skills.

References

Click Here: for link to article used for Human Program's

Appendix

ai_trans.c

```
char transpose_submit_desc[] = "Transpose submission";
void transpose_submit(int M, int N, int A[N][M], int B[M][N]){
* At most 12 local ints in this function */
    int i, j, k, l;
    int a0, a1, a2, a3, a4, a5, a6, a7;
```

```
/* Case 1: 32 x 32 */
if (M == 32 && N == 32) {
    for (i = 0; i < 32; i += 8) {
        for (j = 0; j < 32; j += 8) {
            for (k = 0; k < 8; ++k) {
                /* Row i+k, columns j..j+7 */
                a0 = A[i + k][j + 0];
                a1 = A[i + k][j + 1];
                a2 = A[i + k][j + 2];
                a3 = A[i + k][j + 3];
                a4 = A[i + k][j + 4];
                a5 = A[i + k][j + 5];
                a6 = A[i + k][j + 6];
                a7 = A[i + k][j + 7];

                B[j + 0][i + k] = a0;
                B[j + 1][i + k] = a1;
                B[j + 2][i + k] = a2;
                B[j + 3][i + k] = a3;
                B[j + 4][i + k] = a4;
                B[j + 5][i + k] = a5;
                B[j + 6][i + k] = a6;
                B[j + 7][i + k] = a7;
            }
        }
    }
    return;
}

/* Case 2: 64 x 64 */
if (M == 64 && N == 64) {
    for (i = 0; i < 64; i += 8) {
        for (j = 0; j < 64; j += 8) {

            /* Top 4 rows of the 8x8 block */
            for (k = 0; k < 4; ++k) {
                a0 = A[i + k][j + 0];
                a1 = A[i + k][j + 1];
                a2 = A[i + k][j + 2];
                a3 = A[i + k][j + 3];
                a4 = A[i + k][j + 4];
                a5 = A[i + k][j + 5];
                a6 = A[i + k][j + 6];
                a7 = A[i + k][j + 7];

                /* Left 4x4 goes directly to its place */
```

```
    B[j + 0][i + k] = a0;
    B[j + 1][i + k] = a1;
    B[j + 2][i + k] = a2;
    B[j + 3][i + k] = a3;

    /* Right 4 values go to top of right half (temporary) */
    B[j + 0][i + k + 4] = a4;
    B[j + 1][i + k + 4] = a5;
    B[j + 2][i + k + 4] = a6;
    B[j + 3][i + k + 4] = a7;
}

/* Now handle the bottom 4 rows and do the swap between
 * the top-right and bottom-left 4x4 sub-blocks.
 */
for (k = 0; k < 4; ++k) {
    /* Values from bottom-left sub-block of A */
    a0 = A[i + 4][j + k];
    a1 = A[i + 5][j + k];
    a2 = A[i + 6][j + k];
    a3 = A[i + 7][j + k];

    /* Values previously stored in top-right area of B */
    a4 = B[j + k][i + 4];
    a5 = B[j + k][i + 5];
    a6 = B[j + k][i + 6];
    a7 = B[j + k][i + 7];

    /* Move A bottom-left into B top-right (final place) */
    B[j + k][i + 4] = a0;
    B[j + k][i + 5] = a1;
    B[j + k][i + 6] = a2;
    B[j + k][i + 7] = a3;

    /* Move saved top-right into B bottom-left */
    B[j + 4 + k][i + 0] = a4;
    B[j + 4 + k][i + 1] = a5;
    B[j + 4 + k][i + 2] = a6;
    B[j + 4 + k][i + 3] = a7;
}

/* Finally, bottom-right 4x4 sub-block */
for (k = 4; k < 8; ++k) {
    a0 = A[i + 4][j + k];
    a1 = A[i + 5][j + k];
    a2 = A[i + 6][j + k];
```

```
                    a3 = A[i + 7][j + k];

                    B[j + k][i + 4] = a0;
                    B[j + k][i + 5] = a1;
                    B[j + k][i + 6] = a2;
                    B[j + k][i + 7] = a3;
                }
            }
        }
        return;
    }

    /* Case 3: 61 x 67 and any other "general" sizes
     * Use 16x16 blocking to keep cache behavior decent and handle edges.
     */
    for (i = 0; i < N; i += 16) {
        for (j = 0; j < M; j += 16) {
            for (k = i; k < N && k < i + 16; ++k) {
                for (l = j; l < M && l < j + 16; ++l) {
                    B[l][k] = A[k][l];
                }
            }
        }
    }
}

char trans_desc[] = "Simple row-wise scan transpose";
void trans(int M, int N, int A[N][M], int B[M][N]){
    int i, j, tmp;

    for (i = 0; i < N; i++) {
        for (j = 0; j < M; j++) {
            tmp = A[i][j];
            B[j][i] = tmp;
        }
    }

}

void registerFunctions(){
    /* Register your solution function */
    registerTransFunction(transpose_submit, transpose_submit_desc);

    /* Register any additional transpose functions */
    registerTransFunction(trans, trans_desc);
```

```
}

int is_transpose(int M, int N, int A[N][M], int B[M][N]){
    int i, j;

    for (i = 0; i < N; i++) {
        for (j = 0; j < M; ++j) {
            if (A[i][j] != B[j][i]) {
                return 0;
            }
        }
    }
    return 1;
}
```

human_trans.c

```
void transpose_32_by_32(int M, int N, int A[N][M], int B[M][N]){

    const int BLOCK_SIZE = 8;

    for(int block_row = 0; block_row < N; block_row += BLOCK_SIZE){
        for(int block_column = 0; block_column < M; block_column += BLOCK_SIZE){
            for(int row = block_row; row < block_row + BLOCK_SIZE; row++){
                int diag_tmp = -1;
                int diag_r = -1;
                int diag_c = -1;

                int normalized_row = row;
                if(normalized_row >= BLOCK_SIZE){
                    normalized_row -= block_row;
                }

                for(int column = block_column; column < block_column + BLOCK_SIZE; column++)
                    int normalize_column = column;
                    if(normalize_column >= BLOCK_SIZE){
                        normalize_column -= block_column;
                    }
                    if(normalized_row == normalize_column){
                        diag_tmp = A[row][column];
                        diag_r = row;
                        diag_c = column;
                    }else{
                        int tmp = A[row][column];
                        B[column][row] = tmp;
                    }
```

```
                }
                if(diag_tmp != -1 && diag_r != -1 && diag_c != -1){
                    B[diag_c][diag_r] = diag_tmp;
                }
            }

        }
    }
}

void transpose_64_by_64(int M, int N, int A[N][M], int B[M][N]){
    int t0,t1,t2,t3,t4,t5,t6,t7,row,column;
    for(int block_row = 0; block_row < N; block_row += 8){
        for(int block_column = 0; block_column<M; block_column+=8){
            row = block_row;
            column = block_column;


            for(; row < block_row+4; row++){
                t0 = A[row][column];
                t1 = A[row][column+1];
                t2 = A[row][column+2];
                t3 = A[row][column+3];
                t4 = A[row][column+4];
                t5 = A[row][column+5];
                t6 = A[row][column+6];
                t7 = A[row][column+7];

                B[column][row] = t0;
                B[column+1][row] = t1;
                B[column+2][row] = t2;
                B[column+3][row] = t3;

                B[column][row+4] = t7;
                B[column+1][row+4] = t6;
                B[column+2][row+4] = t5;
                B[column+3][row+4] = t4;
            }

            for(; row < block_row+8; row++){
                t0 = A[row][column];
                t1 = A[row][column+1];
                t2 = A[row][column+2];
                t3 = A[row][column+3];
                t4 = A[row][column+4];
                t5 = A[row][column+5];
```

```
    t6 = A[row][column+6];
    t7 = A[row][column+7];

    B[column+4][row-4] = t3;
    B[column+5][row-4] = t2;
    B[column+6][row-4] = t1;
    B[column+7][row-4] = t0;

    B[column+4][row] = t4;
    B[column+5][row] = t5;
    B[column+6][row] = t6;
    B[column+7][row] = t7;
}

row = block_row;


t0 = B[column+4][row];
t1 = B[column+4][row+1];
t2 = B[column+4][row+2];
t3 = B[column+4][row+3];

t4 = B[column+3][row+4];
t5 = B[column+3][row+5];
t6 = B[column+3][row+6];
t7 = B[column+3][row+7];

B[column+4][row] = t4;
B[column+4][row+1] = t5;
B[column+4][row+2] = t6;
B[column+4][row+3] = t7;

B[column+3][row+4] = t0;
B[column+3][row+5] = t1;
B[column+3][row+6] = t2;
B[column+3][row+7] = t3;

//row 2
t0 = B[column+5][row];
t1 = B[column+5][row+1];
t2 = B[column+5][row+2];
t3 = B[column+5][row+3];

t4 = B[column+2][row+4];
t5 = B[column+2][row+5];
t6 = B[column+2][row+6];
```

```
t7 = B[column+2][row+7];

B[column+5][row] = t4;
B[column+5][row+1] = t5;
B[column+5][row+2] = t6;
B[column+5][row+3] = t7;

B[column+2][row+4] = t0;
B[column+2][row+5] = t1;
B[column+2][row+6] = t2;
B[column+2][row+7] = t3;

//row 3
t0 = B[column+6][row];
t1 = B[column+6][row+1];
t2 = B[column+6][row+2];
t3 = B[column+6][row+3];

t4 = B[column+1][row+4];
t5 = B[column+1][row+5];
t6 = B[column+1][row+6];
t7 = B[column+1][row+7];

B[column+6][row] = t4;
B[column+6][row+1] = t5;
B[column+6][row+2] = t6;
B[column+6][row+3] = t7;

B[column+1][row+4] = t0;
B[column+1][row+5] = t1;
B[column+1][row+6] = t2;
B[column+1][row+7] = t3;

//row 4
t0 = B[column+7][row];
t1 = B[column+7][row+1];
t2 = B[column+7][row+2];
t3 = B[column+7][row+3];

t4 = B[column][row+4];
t5 = B[column][row+5];
t6 = B[column][row+6];
t7 = B[column][row+7];

B[column+7][row] = t4;
B[column+7][row+1] = t5;
```

```
            B[column+7][row+2] = t6;
            B[column+7][row+3] = t7;

            B[column][row+4] = t0;
            B[column][row+5] = t1;
            B[column][row+6] = t2;
            B[column][row+7] = t3;
        }
    }
}

void transpose_61_by_67(int M, int N, int A[N][M], int B[M][N]){
    //12 Local Variables
 int i, j ,k; //Loop iters
 int t0, t1, t2, t3, t4, t5, t6, t7; //Temps for 8, blk size is 8

 const int BLK = 8;

 //Outer Loop for 8x8
 for (i = 0; i < N; i += BLK){
  for (j = 0; j < M; j += BLK){

   //Inner Transposing Loops
   for (k = i; k < i + BLK && k < N; k++){

    //Load a row of A into local at a time to min A accesses, 1 miss per row
    if (j + 0 < M) t0 = A[k][j+0];
    if (j + 1 < M) t1 = A[k][j+1];
    if (j + 2 < M) t2 = A[k][j+2];
    if (j + 3 < M) t3 = A[k][j+3];
    if (j + 4 < M) t4 = A[k][j+4];
    if (j + 5 < M) t5 = A[k][j+5];
    if (j + 6 < M) t6 = A[k][j+6];
    if (j + 7 < M) t7 = A[k][j+7];

    //Store Row into the Same B Col
    if (j + 0 < M) B[j+0][k] = t0;
    if (j + 1 < M) B[j+1][k] = t1;
    if (j + 2 < M) B[j+2][k] = t2;
    if (j + 3 < M) B[j+3][k] = t3;
    if (j + 4 < M) B[j+4][k] = t4;
    if (j + 5 < M) B[j+5][k] = t5;
    if (j + 6 < M) B[j+6][k] = t6;
    if (j + 7 < M) B[j+7][k] = t7;
   }
  }
```

```
 }

}

void transpose_submit(int M, int N, int A[N][M], int B[M][N]){
    if(N == 32 && M == 32){
        transpose_32_by_32(M,N,A,B);
    }else if(M == 64 && N == 64){
        transpose_64_by_64(M,N,A,B);
    }else{
        transpose_61_by_67(M,N,A,B);
    }
}
```